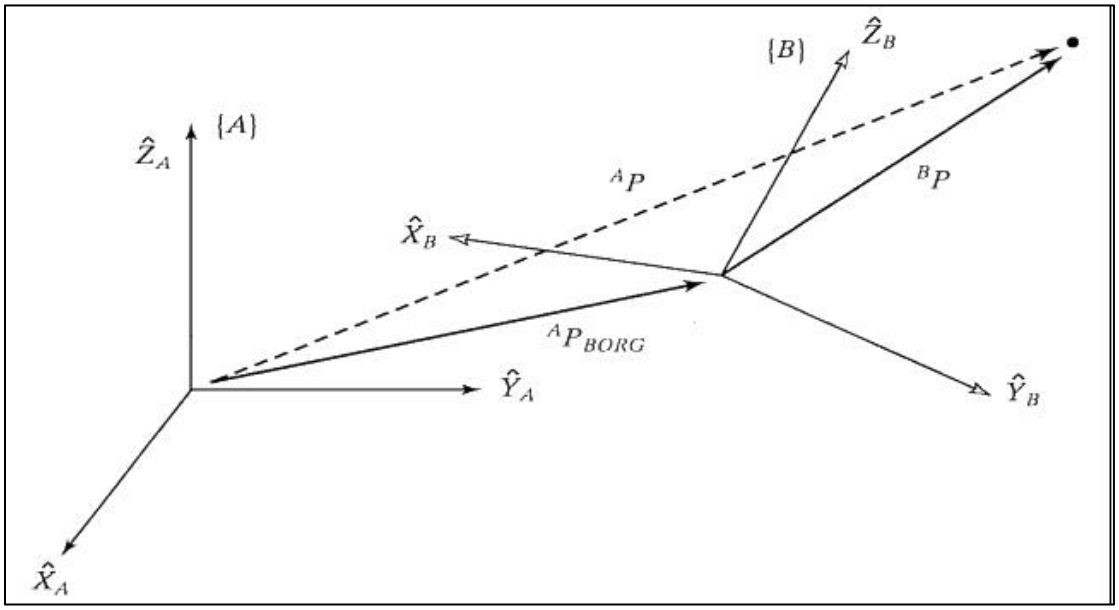
16.ROS2 TF2 机器人坐标系管理器

1.TF2 简介

坐标系是我们非常熟悉的一个概念，也是机器人学中的重要基础，在一个完整的机器人系统中，会存在很多坐标系，这些坐标系之间的位置关系该如何管理？ROS 给我们提供了一个坐标系的管理神奇：TF2。

在移动机器人系统中，坐标系一样至关重要，比如一个移动机器人的中心点是基坐标系 Base Link，雷达所在的位置叫做雷达坐标系 laser link，机器人要移动，里程计会累计位置，这个位置的参考系叫做里程计坐标系 odom，里程计又会有累计误差和飘逸，绝对位置的参考系叫做地图坐标系 map。

一层一层坐标系之间关系复杂，有一些是相对固定的，也有一些是不断变化的，看似简单的坐标系也在空间范围内变得复杂，良好的坐标系管理系统就显得格外重要。



关于坐标系变换关系的基本理论，在每一个机器人学的教材中都会讲解，可以分解为平移和旋转两个部分，通过一个 4*4 的矩阵进行描述，在空间中画出坐标系，那两者之间的变换关系，其实就是向量的教学描述。

ROS 中 TF 功能的底层原理，就是对这些数学变换进行了封装，详细的理论知识大家可

以参考机器人学的教材，我们主要讲解 TF 坐标管理系统的使用方法。

2．TF 命令行操作（测试）

此小点帮助大家快速体验 tf 工具，如果想深入学习可以跳转到“3.静态 TF 广播”进行学习。

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**sudo apt install ros-foxy-turtle-tf2-py**”安装 turtle 的 tf 工具。

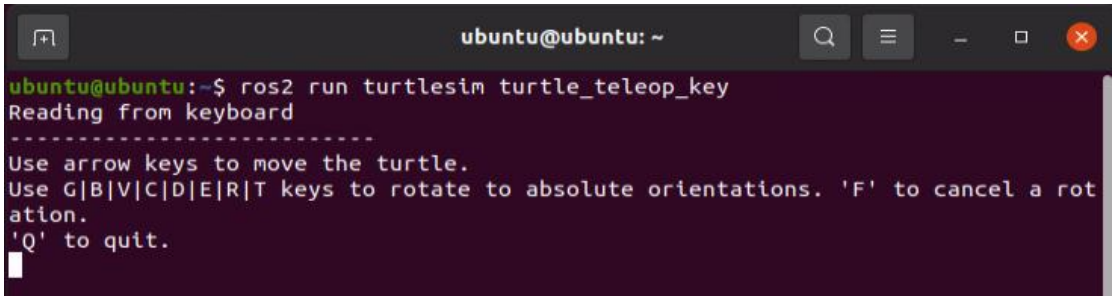
```
ubuntu@ubuntu:~$ sudo apt install ros-foxy-turtle-tf2-py
```

2） 然后接着在终端里面输入指令：

“**ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py**”指令，运行 turtle_tf2_demo.launch.py 文件。

```
ubuntu@ubuntu:~$ ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
```

3） 然后再新建一个命令行终端，接着输入指令“**ros2 run turtlesim turtle_teleop_key**”指令，运行小海龟键盘控制节点。



4） 按“↑↓←→”键控制小海龟移动，另外一只小海龟也会跟着运动。



5） 新建一个终端。输入“**ros2 run tf2_ros tf2_echo turtle2 turtle1**”指令，查看 turtle1 和 turtle2 两个坐标系之间的具体关系。

```
ubuntu@ubuntu:~$ ros2 run tf2_ros tf2_echo turtle2 turtle1
[INFO] [1709038298.386969189] [tf2_echo]: Waiting for transform turtle2 ->  turt
le1: Invalid frame ID "turtle2" passed to canTransform argument target_frame - f
rame does not exist
At time 1709038299.344632416
- Translation: [0.000, 0.000, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.156, 0.988]
At time 1709038300.352673560
- Translation: [0.457, -0.148, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.156, 0.988]
At time 1709038301.344087434
- Translation: [1.195, 0.278, 0.000]
- Rotation: in Quaternion [0.000, 0.000, 0.411, 0.912]
At time 1709038302.353657405
- Translation: [1.669, 0.753, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.076, 0.997]
At time 1709038303.344540712
- Translation: [0.968, 0.202, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.796, 0.605]
```

终端中循环打印坐标系的变换数值，由平移和旋转两个部分组成，还有旋转矩阵。

3.静态 TF 广播

3.1 创建 static_tf_broadcaster.py

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**sudo apt-get install ros-foxy-tf***”安装 tf 工具：

```
ubuntu@ubuntu:~$ sudo apt-get install ros-foxy-tf*
```

2） 然后接着输入指令，“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

3） 输入“**ros2 pkg create tf_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**tf_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create tf_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: tf_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
```

4） 接着在终端里面输入指令“**cd tf_demo/tf_demo/**”指令，切换到 tf_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd tf_demo/tf_demo/
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$
```

5） 输入指令“**gedit static_tf_broadcaster.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ gedit static_tf_broadcaster.py
```

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-









import rclpy
# ROS2 Python 接口库

from rclpy.node import Node
 # ROS2 节点类
```

```
from geometry_msgs.msg import TransformStamped
   # 坐标变换消息
import tf_transformations
 # TF 坐标变换库
from tf2_ros.static_transform_broadcaster import StaticTransformBroadcas
ter  # TF 静态坐标系广播器类


class StaticTFBroadcaster(Node):
    def __init__(self, name):
        super().__init__(name)
    # ROS2 节点父类初始化
        self.tf_broadcaster = StaticTransformBroadcaster(self)
       # 创建一个 TF 广播器对象


        static_transformStamped = TransformStamped()
      # 创建一个坐标变换的消息对象
        static_transformStamped.header.stamp = self.get_clock().now().to
_msg()  # 设置坐标变换消息的时间戳
        static_transformStamped.header.frame_id = 'world'
      # 设置一个坐标变换的源坐标系
        static_transformStamped.child_frame_id  = 'house'
      # 设置一个坐标变换的目标坐标系
        static_transformStamped.transform.translation.x = 10.0
       # 设置坐标变换中的 X、Y、Z 向的平移
```

```
        static_transformStamped.transform.translation.y = 5.0


        static_transformStamped.transform.translation.z = 0.0

        quat = tf_transformations.quaternion_from_euler(0.0, 0.0, 0.0)
       # 将欧拉角转换为四元数（roll, pitch, yaw）

        static_transformStamped.transform.rotation.x = quat[0]
       # 设置坐标变换中的 X、Y、Z 向的旋转（四元数）

        static_transformStamped.transform.rotation.y = quat[1]

        static_transformStamped.transform.rotation.z = quat[2]

        static_transformStamped.transform.rotation.w = quat[3]


        self.tf_broadcaster.sendTransform(static_transformStamped)
       # 广播静态坐标变换，广播后两个坐标系的位置关系保持不变


def main(args=None):
    rclpy.init(args=args)                                   # ROS2 Python 接口
初始化
    node = StaticTFBroadcaster("static_tf_broadcaster")  # 创建 ROS2 节点对
象并进行初始化
    rclpy.spin(node)                                        # 循环等待 ROS2 退出
    node.destroy_node()                                     # 销毁节点对象
    rclpy.shutdown()
```

6） 输入指令“**chmod +x static_tf_broadcaster.py**”，并按下回车，为保存的

static_tf_broadcaster.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ chmod +x static_tf_broadcaster.py
```

3.2 创建 tf_listener.py

1) 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**gedit tf_listener.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ gedit tf_listener.py
```

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-

import rclpy
# ROS2 Python 接口库

from rclpy.node import Node
 # ROS2 节点类

from geometry_msgs.msg import TransformStamped
   # 坐标变换消息

import tf_transformations
 # TF 坐标变换库

from tf2_ros.static_transform_broadcaster import StaticTransformBroadcas
ter  # TF 静态坐标系广播器类



class StaticTFBroadcaster(Node):

    def __init__(self, name):
```

```
        super().__init__(name)
    # ROS2 节点父类初始化
        self.tf_broadcaster = StaticTransformBroadcaster(self)
       # 创建一个 TF 广播器对象


        static_transformStamped = TransformStamped()
      # 创建一个坐标变换的消息对象
        static_transformStamped.header.stamp = self.get_clock().now().to
_msg()  # 设置坐标变换消息的时间戳
        static_transformStamped.header.frame_id = 'world'
      # 设置一个坐标变换的源坐标系
        static_transformStamped.child_frame_id  = 'home'
     # 设置一个坐标变换的目标坐标系
        static_transformStamped.transform.translation.x = 6.0
     # 设置坐标变换中的 X、Y、Z 向的平移
        static_transformStamped.transform.translation.y = 11.0


        static_transformStamped.transform.translation.z = 0.0
        quat = tf_transformations.quaternion_from_euler(0.0, 0.0, 0.0)
       # 将欧拉角转换为四元数（roll, pitch, yaw）
        static_transformStamped.transform.rotation.x = quat[0]
       # 设置坐标变换中的 X、Y、Z 向的旋转（四元数）
        static_transformStamped.transform.rotation.y = quat[1]
        static_transformStamped.transform.rotation.z = quat[2]
```

```
        static_transformStamped.transform.rotation.w = quat[3]



        self.tf_broadcaster.sendTransform(static_transformStamped)
       # 广播静态坐标变换，广播后两个坐标系的位置关系保持不变


def main(args=None):
    rclpy.init(args=args)                                   # ROS2 Python 接口
初始化
    node = StaticTFBroadcaster("static_tf_broadcaster")  # 创建 ROS2 节点对
象并进行初始化
    rclpy.spin(node)                                       # 循环等待 ROS2 退出
    node.destroy_node()                                    # 销毁节点对象
    rclpy.shutdown()
```

2） 输入指令“**chmod +x tf_listener.py**”，并按下回车，为保存的 tf_listener.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ chmod +x tf_listener.py
```

3.3 setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 **static_tf_broadcaster.py** 和 **tf_listener.py** 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ cd ..
```

2） 输入“**geditsetup.py**”指令，并按下回车，打开 setup.py 文件，复制下面程序，

使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo$ gedit setup.py
```

```
'static_tf_broadcaster = tf_demo.static_tf_broadcaster:main',

'tf_listener = tf_demo.tf_listener:main',
```

```
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='ubuntu',
17     maintainer_email='ubuntu@todo.todo',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'static_tf_broadcaster = tf_demo.static_tf_broadcaster:main',
24             'tf_listener = tf_demo.tf_listener:main',
25         ],
26     },
27 )
```

3.4 编译运行

1） 新建一个命令行终端，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run tf_demo static_tf_broadcaster**”，并按下回车，启动 static_tf_broadcaster 节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run tf_demo static_tf_broadcaster
```

5） 然后再新建一个新的终端窗口，输入指令“**ros2 run tf_demo tf_listener**”，

并按下回车，启动 tf_listener 监听。

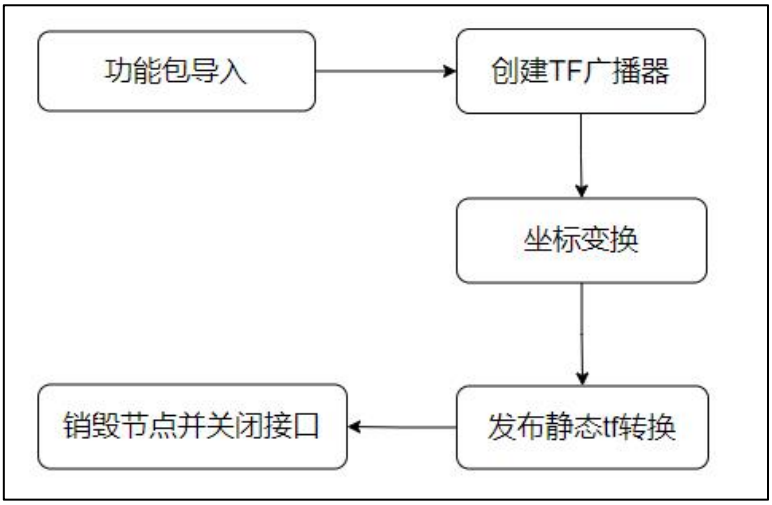
```
ubuntu@ubuntu:~$ ros2 run tf_demo tf_listener
[INFO] [1709039619.017241471] [tf_listener]: Get world --> home transform: [-6.000000,
 -11.000000, 0.000000] [0.000000, -0.000000, 0.000000]
[INFO] [1709039619.987198795] [tf_listener]: Get world --> home transform: [-6.000000,
 -11.000000, 0.000000] [0.000000, -0.000000, 0.000000]
[INFO] [1709039620.987166775] [tf_listener]: Get world --> home transform: [-6.000000,
 -11.000000, 0.000000] [0.000000, -0.000000, 0.000000]
[INFO] [1709039621.987473128] [tf_listener]: Get world --> home transform: [-6.000000,
 -11.000000, 0.000000] [0.000000, -0.000000, 0.000000]
[INFO] [1709039622.987546805] [tf_listener]: Get world --> home transform: [-6.000000,
 -11.000000, 0.000000] [0.000000, -0.000000, 0.000000]
```

可以看到当前系统中存在两个坐标系，一个是 world，一个是 house，两者之间的相对位置不会发生改变，通过一个静态的 TF 对象进行维护。

3.5 程序分析

3.5.1 static_tf_broadcaster.py

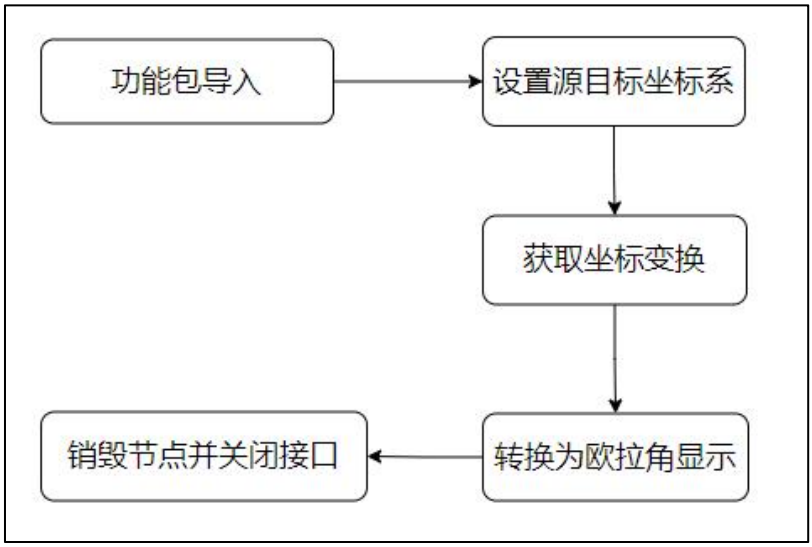
根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了 StaticTFBroadcaster 节点类,初始化节点对象时创建了 TF 广播器,然后生成包含源坐标系世界坐标和目标 home 坐标以及转换信息如平移旋转等的坐标转换消息,利用 tf_transformations 将欧拉角转换成四元数格式,在节点运行期间通过广播器对象持续广播这个静态坐标转换信息,目的是为其他节点或工具提供从世界坐标到 home 坐标的坐标变换支持。

3.5.2 tf_listener.py

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了 TFListener 节点类，初始化节点对象时创建了 TF 缓冲区和监听器，设置了动态获取源目标坐标系的参数，通过定时器接口定期从监听器查询指定坐标系间的坐标转换关系，获取其位置姿态信息打印到日志，同时处理了坐标转换失败的异常情况，其作用是实时监测指定坐标系间的动态转换关系，并为其他节点或用户提供实时的坐标转换支持。

4.TF 监听

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/tf_demo/tf_demo/**”指令，切换到 tf_demo 的文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/tf_demo/tf_demo/
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$
```

1） 输入指令“**gedit turtle_tf_broadcaster.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ gedit turtle_tf_broadcaster.py
```

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-









import rclpy                                        # ROS2 Python 接口库

from rclpy.node import Node                         # ROS2 节点类

from geometry_msgs.msg import TransformStamped      # 坐标变换消息

import tf_transformations                           # TF 坐标变换库

from tf2_ros import TransformBroadcaster            # TF 坐标变换广播器

from turtlesim.msg import Pose                      # turtlesim 小海龟位置消息
```

```
class TurtleTFBroadcaster(Node):



    def __init__(self, name):

        super().__init__(name)                                  # ROS2 节点父类
初始化



        self.declare_parameter('turtlename', 'turtle')          # 创建一个海
龟名称的参数

        self.turtlename = self.get_parameter(                   # 优先使用外
部设置的参数值，否则用默认值

            'turtlename').get_parameter_value().string_value



        self.tf_broadcaster = TransformBroadcaster(self)        # 创建一个 T
F 坐标变换的广播对象并初始化



        self.subscription = self.create_subscription(           # 创建一个订
阅者，订阅海龟的位置消息

            Pose,

            f'/{self.turtlename}/pose',                         # 使用参数中获
取到的海龟名称

            self.turtle_pose_callback, 1)



    def turtle_pose_callback(self, msg):                                # 创
建一个处理海龟位置消息的回调函数，将位置消息转变成坐标变换
```

```
        transform = TransformStamped()                                  # 创建
一个坐标变换的消息对象



        transform.header.stamp = self.get_clock().now().to_msg()      #
设置坐标变换消息的时间戳

        transform.header.frame_id = 'world'                             # 设
置一个坐标变换的源坐标系

        transform.child_frame_id = self.turtlename                      # 设
置一个坐标变换的目标坐标系

        transform.transform.translation.x = msg.x                       # 设
置坐标变换中的 X、Y、Z 向的平移

        transform.transform.translation.y = msg.y

        transform.transform.translation.z = 0.0

        q = tf_transformations.quaternion_from_euler(0, 0, msg.theta) #
将欧拉角转换为四元数（roll, pitch, yaw）

        transform.transform.rotation.x = q[0]                           # 设
置坐标变换中的 X、Y、Z 向的旋转（四元数）

        transform.transform.rotation.y = q[1]

        transform.transform.rotation.z = q[2]

        transform.transform.rotation.w = q[3]



        # Send the transformation

        self.tf_broadcaster.sendTransform(transform)     # 广播坐标变换，海
龟位置变化后，将及时更新坐标变换信息
```

```
def main(args=None):
    rclpy.init(args=args)                                   # ROS2 Python 接口初始化
    node = TurtleTFBroadcaster("turtle_tf_broadcaster")  # 创建 ROS2 节点对象并进行初始化
    rclpy.spin(node)                                        # 循环等待 ROS2 退出
    node.destroy_node()                                     # 销毁节点对象
    rclpy.shutdown()                                        # 关闭 ROS2 Python 接口
```

2） 输入指令“**chmod +x turtle_tf_broadcaster.py**”，并按下回车，为保存的 turtle_tf_broadcaster.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ chmod +x turtle_tf_broadcaster.py
```

4.2 创建 turtle_following.py

3） 在当前的终端里面，输入指令“**gedit turtle_following.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ gedit turtle_following.py
```

```
    #!/usr/bin/env python3
# -*- coding: utf-8 -*-




import math
```

```
import rclpy                                                    # ROS2 Python 接口
库
from rclpy.node import Node                                     # ROS2 节点类
import tf_transformations                                       # TF 坐标变换库
from tf2_ros import TransformException                          # TF 左边变换的异
常类
from tf2_ros.buffer import Buffer                               # 存储坐标变换信息
的缓冲类
from tf2_ros.transform_listener import TransformListener  # 监听坐标变换
的监听器类
from geometry_msgs.msg import Twist                             # ROS2 速度控制消
息
from turtlesim.srv import Spawn                                 # 海龟生成的服务接
口
class TurtleFollowing(Node):


    def __init__(self, name):
        super().__init__(name)                                  # ROS2 节
点父类初始化


        self.declare_parameter('source_frame', 'turtle1')       # 创建
一个源坐标系名的参数
        self.source_frame = self.get_parameter(                 # 优先
使用外部设置的参数值，否则用默认值
```

```
            'source_frame').get_parameter_value().string_value


        self.tf_buffer = Buffer()                                    # 创建保
存坐标变换信息的缓冲区
        self.tf_listener = TransformListener(self.tf_buffer, self)  # 创
建坐标变换的监听器


        self.spawner = self.create_client(Spawn, 'spawn')            # 创建
一个请求产生海龟的客户端
        self.turtle_spawning_service_ready = False                   # 是否
已经请求海龟生成服务的标志位
        self.turtle_spawned = False                                  # 海龟是
否产生成功的标志位


        self.publisher = self.create_publisher(Twist, 'turtle2/cmd_vel',
1) # 创建跟随运动海龟的速度话题


        self.timer = self.create_timer(1.0, self.on_timer)          # 创建
一个固定周期的定时器，控制跟随海龟的运动


    def on_timer(self):
        from_frame_rel = self.source_frame                           # 源坐标
系
        to_frame_rel   = 'turtle2'                                   # 目标坐
```

```
标系


        if self.turtle_spawning_service_ready:                    # 如果已
经请求海龟生成服务
            if self.turtle_spawned:                               # 如果跟
随海龟已经生成
                try:
                    now = rclpy.time.Time()                       # 获取 ROS
系统的当前时间
                    trans = self.tf_buffer.lookup_transform(      # 监听当
前时刻源坐标系到目标坐标系的坐标变换
                        to_frame_rel,
                        from_frame_rel,
                        now)
                except TransformException as ex:                  # 如果坐
标变换获取失败，进入异常报告
                    self.get_logger().info(
                        f'Could not transform {to_frame_rel} to {from_fram
e_rel}: {ex}')
                    return

                msg = Twist()                                     # 创建速度
控制消息
                scale_rotation_rate = 1.0                         # 根据海
```

```
龟角度，计算角速度
                msg.angular.z = scale_rotation_rate * math.atan2(
                    trans.transform.translation.y,
                    trans.transform.translation.x)

                scale_forward_speed = 0.5                           # 根据海
龟距离，计算线速度
                msg.linear.x = scale_forward_speed * math.sqrt(
                    trans.transform.translation.x ** 2 +
                    trans.transform.translation.y ** 2)

                self.publisher.publish(msg)                         # 发布速
度指令，海龟跟随运动
            else:                                                   # 如果跟随
海龟没有生成
                if self.result.done():                              # 查看海
龟是否生成
                    self.get_logger().info(
                        f'Successfully spawned {self.result.result().name}
')
                    self.turtle_spawned = True
                else:                                               # 依然没有
生成跟随海龟
```

```
                self.get_logger().info('Spawn is not finished')

        else:                                                    # 如果没有
请求海龟生成服务

            if self.spawner.service_is_ready():                  # 如果海
龟生成服务器已经准备就绪

                request = Spawn.Request()                        # 创建一
个请求的数据

                request.name = 'turtle2'                         # 设置请
求数据的内容，包括海龟名、xy 位置、姿态

                request.x = float(4)

                request.y = float(2)

                request.theta = float(0)



                self.result = self.spawner.call_async(request)     # 发送
服务请求

                self.turtle_spawning_service_ready = True          # 设置标
志位，表示已经发送请求

            else:

                self.get_logger().info('Service is not ready')     # 海龟
生成服务器还没准备就绪的提示




def main(args=None):

    rclpy.init(args=args)                          # ROS2 Python 接口初始化
```

```
    node = TurtleFollowing("turtle_following")  # 创建 ROS2 节点对象并进行初始化
    rclpy.spin(node)                            # 循环等待 ROS2 退出
    node.destroy_node()                         # 销毁节点对象
    rclpy.shutdown()                            # 关闭 ROS2 Python 接口
```

4） 输入指令“**chmod +x turtle_following.py**”，并按下回车，为保存的 turtle_following.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ chmod +x turtle_following.py
```

4.3 创建 turtle_following.launch.py

在这个例程我们需要开启小海龟仿真器、海龟 1 的坐标系广播、海龟 2 的坐标系广播和海龟跟随控制的四个节点，可以将开启这四个节点的指令写在同一个 launch 文件中，运行 launch 文件即可完成四个节点开启。

1） 输入“**cd ..**”指令，返回上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/tf_demo$ cd ..
```

2） 然后输入“**mkdir launch**”指令，创建 launch 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo$ mkdir launch
```

3） 输入“**cd launch/**”指令，进入 launch 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo$ cd launch/
```

4） 输入指令“**gedit turtle_following.launch.py**”编辑程序，复制下面程序，使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/tf_demo/launch$ gedit turtle_following.launch.py
```

```
from launch import LaunchDescription

from launch.actions import DeclareLaunchArgument

from launch.substitutions import LaunchConfiguration

from launch_ros.actions import Node



def generate_launch_description():

    return LaunchDescription([

        Node(

            package='turtlesim',

            executable='turtlesim_node',

            name='sim'

        ),

        Node(

            package='tf_demo',

            executable='turtle_tf_broadcaster',

            name='broadcaster1',

            parameters=[

                {'turtlename': 'turtle1'}

            ]

        ),

        DeclareLaunchArgument(
```

```
        'target_frame', default_value='turtle1',
        description='Target frame name.'
    ),
    Node(
        package='tf_demo',
        executable='turtle_tf_broadcaster',
        name='broadcaster2',
        parameters=[
            {'turtlename': 'turtle2'}
        ]
    ),
    Node(
        package='tf_demo',
        executable='turtle_following',
        name='listener',
        parameters=[
            {'target_frame': LaunchConfiguration('target_frame')}
        ]
    ),
])
```

5） 输入指令“**chmod +x turtle_following.launch.py**”，并按下回车，为保存的 turtle_following.launch.py 赋予可执行权限。



4.3 setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 **turtle_tf_broadcaster.py** 和 **turtle_following.py** 的程序入口写到 setup.py 文件中。

3） 输入“**cd ..**”指令，切换到上一级目录。



4） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。



输入以下代码到相应的位置上。

```
import os

from glob import glob

(os.path.join('share', package_name, 'launch'), glob(os.path.join('launc
h', '*.launch.py'))),

'turtle_tf_broadcaster = tf_demo.turtle_tf_broadcaster:main',

'turtle_following = tf_demo.turtle_following:main',
```

```
from setuptools import setup
import os
from glob import glob
package_name = 'tf_demo'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*.launch.py'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ubuntu',
    maintainer_email='ubuntu@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
          'static_tf_broadcaster = tf_demo.static_tf_broadcaster:main',
          'tf_listener = tf_demo.tf_listener:main',
          'turtle_tf_broadcaster = tf_demo.turtle_tf_broadcaster:main',
          'turtle_following = tf_demo.turtle_following:main',
        ],
    },
)
```

4.4 编译运行

1） 新建一个命令行终端，然后输入“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

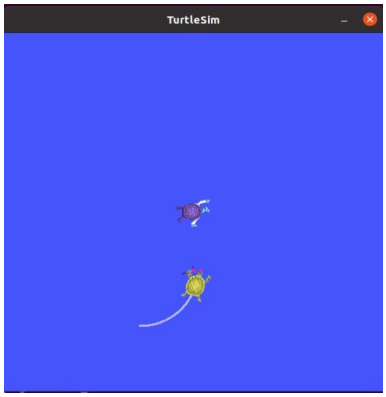
```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch tf_demo turtle_following.launch.py** ”，并按下回车，启动 turtle_fllowing.launch.py 文件。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 launch tf_demo turtle_following.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02-27-05-28-33-792785-ubuntu-20410
```



5） 然后再新建一个新的终端窗口，输入指令“**ros2 run turtlesim turtle_teleop_key**”，并按下回车，启动 turtle_teleop_key 控制节点。

```
ubuntu@ubuntu:~$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rot
ation.
'Q' to quit.
```

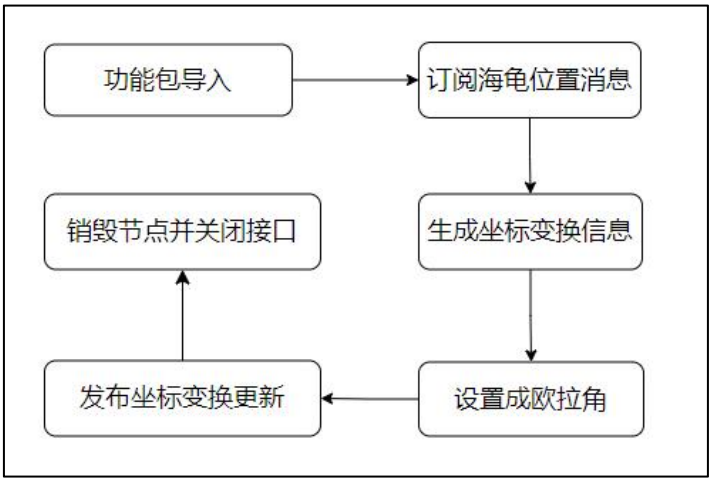
按“↑↓←→”键控制小海龟移动，另外一只小海龟也会跟着运动。



4.5 程序分析

4.5.1 turtle_tf_broadcaster.py

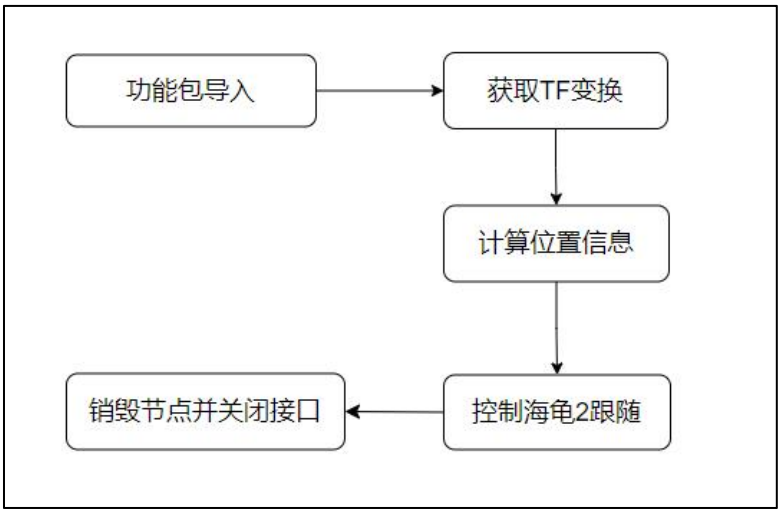
根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了 TurtleTFBroadcaster 节点类，初始化节点对象时创建 TF 广播器，进而订阅小海龟位置消息，在位置回调函数中根据消息生成世界坐标到海龟坐标的坐标变换消息，设置平移为位置，旋转利用欧拉角转换成四元数，并通过广播器实时将坐标变换信息传播出去，从而能根据小海龟实时位置动态更新它在世界坐标下的坐标变换关系，为后续规划控制等节点和用户提供小海龟实时坐标转换支持。

4.5.2 turtle_following.py

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了 TurtleFollowing 节点类，初始化节点后创建了 TF 监听器和速度 publisher，通过定期查询 TF 获取源海龟与目标海龟的坐标关系，计算出目标海龟的速度指令值，第一步请求生成跟随海龟，然后按源海龟的位置和姿态实时控制目标海龟运动实现跟随效果，从而基于 TF 坐标变换关系实现了 ROS2 环境下两个海龟间的跟随控制应用。